

AI Vibe Coding 기반 소프트웨어 개발 프로세스

테크 다크(네온) 스타일의 14단계 현대적 프로세스

Plan First, Code with AI, Verify Always

```
> System.init();  
> Date: 2026-05-16  
> Status: READY
```

```
vibe-coding-process.ts  
  
import { AI_Vibe_Dev } from '@modern/process';  
  
const project = new AI_Vibe_Dev({  
  phases: 4,  
  steps: 14,  
  theme: "Tech Dark",  
  tools: ["Cursor", "Claude", "Copilot"]  
});  
  
// Initialize full cycle from planning to ops  
await project.executeAllPhases();  
■  
console.log("🚀 Development Successfully  
Initialized");
```

— TABLE OF CONTENTS

/src/docs/process-phases.ts

PHASE 01

기획 & 조사

Planning & Research

- 01 목적/조사
- 02 1차 설계
- 03 설계 보강+SRS

PHASE 02

설계 & 준비

Design & Preparation

- 04 보안·NFR
- 05 UI/UX
- 06 AI 핸드오프 자료
- 07 TDD
- 08 개발환경 세팅

PHASE 03

개발 & 검증

Build & Validate

- 09 AI 바이프 코딩
- 10 반복개발&
프롬프트 튜닝
- 11 검증(QA)

PHASE 04

배포 & 운영

Deploy & Operate

- 12 배포·운영
- 13 최종 보고서
- 14 유지보수·피드백

프로세스 개요 타임라인

/src/diagrams/14-steps-overview.ts



// DELIVERABLE CHAIN

Vision → SRS → Arch/NFR → UI Kit → Test Plan → Dev Env → Increments → Reports → Runbook

PHASE 01

기획 & 조사

문제-해결 적합성을 검증하고,
프로젝트 범위를 확정하며,
초기 아키텍처 방향성을 수립합니다.

```
const phase = {  
  id: "01",  
  status: "INITIALIZING",  
  target: "Architecture Vision"  
};
```

// OBJECTIVE

목표

- 문제-해결 적합성 검증
- MVP 스코프(범위) 확정
- 초기 아키텍처 방향성 수립
- 비즈니스 목표 얼라인먼트

// REQUIREMENTS

입력 (Input)

- 비즈니스 핵심 목표
- 이해관계자 인터뷰 결과
- 시장 및 경쟁사 분석 자료
- 가용 예산 및 일정 제약

// DELIVERABLES

출력 (Output)

- 제품 비전 (Product Vision)
- 사용자 페르소나 / JTBD
- 요구사항 명세서(SRS) 초안
- 리스크 레지스터 (Risk Register)

// VALIDATION

성공 기준

- 해결할 문제 정의의 명확화
- 프로젝트 범위/제약 사항 합의 완수
- 측정 가능한 성공 목표(OKR) 설정
- 주요 이해관계자 최종 승인(Sign-off)

1

PHASE 01 / STEP 01

개발 목적 정의 & 시장 조사

// CORE_ACTIVITIES

핵심 활동

- 문제 정의 (Problem Statement)
- 타겟 세그먼트 및 페르소나(Persona) 도출
- JTBD (Jobs-to-be-Done) 분석
- 경쟁사 및 대체재 분석, 핵심 가치제안 확립

// RECOMMENDED_TOOLS

추천 도구

- Lean Canvas / Business Model Canvas
- SWOT Analysis Tools (Miro, FigJam)
- 설문 및 인터뷰 (Typeform, User Interviews)
- Market Analytics (Google Analytics, Mixpanel)

// OUTPUT_DELIVERABLES

산출물

- 제품 비전 선언문 (Product Vision)
- 핵심 목표 지표 (North Star Metric)
- 초기 스코프 및 비즈니스 가설 목록

// VALIDATION_CHECKPOINTS

체크포인트

- 명확하고 측정 가능한 성공 지표가 정의되었는가?
- 금지 요건 및 법적/규제 제약사항을 파악했는가?
- 초기 ROI(투자 대비 효용) 가설이 합리적인가?

1차 개발 설계

// CORE_ACTIVITIES

핵심 활동

- 도메인 모델링
- 아키텍처 후보 검토 (C4 / Hexagonal)
- 기술스택 옵션 비교 및 선정
- 인터페이스 식별

// RECOMMENDED_TOOLS

추천 도구

- Miro
- Draw.io
- C4 Model
- ADR 템플릿

// OUTPUT_DELIVERABLES

산출물

- 아키텍처 스케치
- ADR (Architecture Decision Record)
- 인터페이스 리스트

// VALIDATION_CHECKPOINTS

체크포인트

- 아키텍처의 변경 용이성이 고려되었는가?
- 리스크가 높은 영역이 명확히 표식되었는가?
- PoC(개념 증명)가 필요한 기술 요소가 있는가?

설계 보강 & 문제점 체크 (SRS 작성)

// CORE_ACTIVITIES

핵심 활동

- 요구사항 명세 (SRS: 기능/비기능/제약사항 통합)
- 프로젝트 리스크 및 잠재적 가정(Assumptions) 명시
- 수용 기준 (Acceptance Criteria, AC) 명확화

// RECOMMENDED_TOOLS

추천 도구

- 협업 문서화 도구 (Confluence, Notion)
- RFC (Request for Comments) 문서 포맷
- IEEE 830 / 29148 표준 SRS 템플릿

// OUTPUT_DELIVERABLES

산출물

- **[추가 권장 핵심]** 요구사항 명세서 (SRS v1)
- 리스크 레지스터 (위험 요소 및 대응 방안 목록)
- MoSCoW 우선순위 맵 (Must/Should/Could/Won't)

// VALIDATION_CHECKPOINTS

체크포인트

- 요구사항의 모호성이 완벽하게 제거되었는가?
- 모든 요구사항에 대해 테스트 가능성(Testability)이 확보되었는가?
- 설계와 범위에 대해 핵심 이해관계자의 서명 및 합의가 완료되었는가?

PHASE 02

설계 & 준비

보안, 성능, 확장성을 포함한 종합 설계를 진행하고, AI 바이브 코딩을 위한 모든 개발 및 테스트 준비를 완료합니다.

```
const phase = {  
  id: "02",  
  status: "DESIGNING",  
  target: "Dev Readiness 100%"  
};
```

// OBJECTIVE

목표

- 보안/성능/확장성 종합 설계
- AI 기반 개발 환경 셋업
- TDD 기반 테스트 준비 완비
- 개발을 위한 UI/UX 확정

// REQUIREMENTS

입력 (Input)

- 요구사항 명세서 (SRS v1)
- 초기 아키텍처 스케치
- 기능 우선순위 백로그
- 제품 비전 및 제약 사항

// DELIVERABLES

출력 (Output)

- 보안 / NFR 설계 문서
- UI/UX 화면 산출물
- 테스트 전략 및 환경
- 코드 생성 AI 핸드오프 자료

// VALIDATION

성공 기준

- 품질속성 달성전략 수립 완료
- AI 핸드오프용 자료 100% 준비
- 보안 요구사항 아키텍처 반영
- 테스트 파이프라인 작동 확인

2차 보안 & 비기능 요구사항 설계

// CORE_ACTIVITIES

핵심 활동

- 위협모델링 (STRIDE)
- 인증 및 인가 체계 설계
- Secrets 및 키(Key) 관리 전략
- 성능 예산, 확장성 및 가용성 설계

// RECOMMENDED_TOOLS

추천 도구

- OWASP ASVS (Application Security Verification Standard)
- OWASP Threat Dragon
- Data Flow Diagram Tools

// OUTPUT_DELIVERABLES

산출물

- 보안 설계 문서
- NFR 스펙 (SLO/SLI 정의)
- 데이터 보호 및 암호화 정책

// VALIDATION_CHECKPOINTS

체크포인트

- 민감 데이터 흐름 및 저장 경로가 파악되었는가?
- 최소 권한의 원칙이 설계에 반영되었는가?
- 장애 및 데이터 유실 시나리오 대비책이 마련되었는가?

UI/UX 디자인 & 화면 설계

// CORE_ACTIVITIES

핵심 활동

- 정보구조(IA) 및 사용자 흐름(User Flow) 정의
- 화면 와이어프레임 설계 및 시각화
- 디자인 시스템 및 재사용 가능한 컴포넌트 구축
- 웹 접근성(WCAG AA) 및 사용성 가이드라인 고려

// OUTPUT_DELIVERABLES

산출물

- 화면 흐름도 (Screen Flow Map)
- 컴포넌트 라이브러리 및 디자인 토큰 가이드
- 클릭 가능한 인터랙션 프로토타입

// RECOMMENDED_TOOLS

추천 도구

- Figma / FigJam (UI 디자인 및 협업)
- Storybook (UI 컴포넌트 라이브러리 개발)
- Axe / Lighthouse (접근성 및 성능 테스트)
- Adobe XD / Sketch

// VALIDATION_CHECKPOINTS

체크포인트

- 핵심 태스크 완료 시 '3클릭 규칙'을 만족하는가?
- 반응형 웹 디자인 및 국제화(다국어 레이아웃)가 고려되었는가?
- 개발자 핸드오프(Handoff)를 위한 규칙과 명세가 명확한가?

AI 전달용 자료 정리 (Context Engineering)

// CORE_ACTIVITIES

핵심 활동

- 문제 컨텍스트 패키징 및 API 스펙 (OpenAPI) 정의
- 샘플 데이터 및 에지 케이스(Edge Cases) 도출
- AI 프롬프트 스타일가이드 작성
- 기술 스택 확정(Python/TS/Go) 및 디자인 패턴 결정

// RECOMMENDED_TOOLS

추천 도구

- OpenAPI / Swagger
- Postman
- Mermaid 다이어그램
- Markdown 문서 에디터

// OUTPUT_DELIVERABLES

산출물

- AI 개발용 브리프 (Context Package)
- 시스템 및 시퀀스 다이어그램
- 데이터 계약서 (Data Contract)

// VALIDATION_CHECKPOINTS

체크포인트

- 문서의 최신성과 일관성이 완벽하게 유지되는가?
- 민감한 개인정보 및 사내 기밀이 마스킹되었는가?
- AI가 재현 가능한 명확한 예제가 포함되어 있는가?

TDD — 개발 전 테스트 케이스

// CORE_ACTIVITIES

핵심 활동

- 테스트 전략 수립 (Unit / Integration / E2E)
- Gherkin 문법 기반 인수 조건(AC) 작성
- 경계값 분석 및 에러 시나리오 도출
- Red → Green → Refactor 사이클 준수
- 목표 코드 커버리지(Coverage) 설정

// OUTPUT_DELIVERABLES

산출물

- 종합 테스트 계획서 (Test Plan)
- 테스트 스펙 및 피처(Feature) 파일
- 모의(Mock) 객체 및 테스트용 샘플 데이터

// RECOMMENDED_TOOLS

추천 도구

- Jest / PyTest / JUnit
- Cypress / Playwright (E2E Testing)
- Pact (Contract Testing)
- Cucumber / Behave (BDD Frameworks)

// VALIDATION_CHECKPOINTS

체크포인트

- 개발 전 실패하는 테스트를 먼저 작성했는가?
- 테스트가 외부 의존성 없이 독립적이고 재현 가능한가?
- CI 환경에서 빠르고 병렬 실행이 가능하도록 설계되었는가?

개발 환경 세팅 (보강)

// CORE_ACTIVITIES

핵심 활동

- 레포지토리 초기화 및 브랜치 전략 수립 (GitFlow/Trunk)
- Dev Container 및 Docker 환경 구성
- CI 파이프라인 템플릿 작성 및 안전한 시크릿 관리
- AI 코딩 도구 셋업 (Cursor, GitHub Copilot 등)

// OUTPUT_DELIVERABLES

산출물

- 부트스트랩(Bootstrap)된 저장소
- 초기 베이스 스캐폴딩(Scaffolding) 코드
- 첫 CI 빌드 성공 (Green) 상태

// RECOMMENDED_TOOLS

추천 도구

- GitHub Actions, GitLab CI
- Docker / Docker Compose
- pre-commit, Makefile, Renovate
- AI Tools: Cursor, Claude Code

// VALIDATION_CHECKPOINTS

체크포인트

- 어디서든 동일하게 재현 가능한 빌드 환경인가?
- 린터 및 포매터(Linter/Formatter) 자동화가 적용되어 있는가?
- 코드 스멜 방지 및 커버리지 유지를 위한 품질 게이트가 있는가?

PHASE 03

개발 & 검증

AI와의 페어프로그래밍으로
기능 증분(Increment)을 개발하고,
엄격한 품질 검증을 통과합니다.

```
const phase = {  
  id: "03",  
  status: "BUILDING",  
  target: "Working Increment"  
};
```

// OBJECTIVE

목표

- AI 기반 페어 프로그래밍
- 기능 증분(Increment) 개발
- 품질 검증 프로세스 통과
- 배포 가능한 상태 확보

// REQUIREMENTS

입력 (Input)

- 작성된 테스트 케이스 (TDD)
- 아키텍처 및 화면 설계 산출물
- 세팅 완료된 개발 환경
- 프롬프트 스타일가이드

// DELIVERABLES

출력 (Output)

- 동작하는 소프트웨어 증분
- Pull Request / 코드리뷰 기록
- 테스트 결과 리포트
- 프롬프트 튜닝 라이브러리

// VALIDATION

성공 기준

- 설정된 스프린트 목표 달성
- 결함 밀도(Defect Density) 기준 만족
- 리드타임(Lead Time) 목표치 달성
- 모든 테스트 및 품질 게이트 통과

9단계: AI 바이브 코딩 시작

// CORE_ACTIVITIES

핵심 활동

- 작업 슬라이싱 (티켓/태스크 단위 세분화)
- 프롬프트 루프 (초안 → 실행 → 리뷰 → 수정)
- AI 코드리뷰 및 적절한 커밋 단위 유지
- Vibe Coding 사이클 (프롬프트 → 생성 → 검토 → 수정)

// OUTPUT_DELIVERABLES

산출물

- 동작하는 기능 증분 (Working Increment)
- PR 및 AI 페어 프로그래밍 코드리뷰 기록
- 재사용 가능한 프롬프트 및 스니펫 라이브러리

// RECOMMENDED_TOOLS

추천 도구

- Cursor, Windsurf (AI 네이티브 IDE)
- VS Code + GitHub Copilot
- Claude Code / Aider (CLI 에이전트)
- GitHub PR Templates

// VALIDATION_CHECKPOINTS

체크포인트

- 테스트 주도 흐름(TDD)을 올바르게 유지하고 있는가?
- 보안 린트 검증 및 취약점 검사를 통과했는가?
- 주요 구조 변경 시 결정 근거(ADR)를 작성했는가?

9+

PHASE 03 / DEEP DIVE

CLEAR 프레임워크로 프롬프트 최적화

C

Context (맥락)

배경, 목적, 아키텍처 및 도메인 정보 제공

L

Language (언어/제약)

프로그래밍 언어, 프레임워크, 라이브러리 명시

E

Examples (입출력 샘플)

정확한 입력 데이터 구조 및 기대 출력 형태

A

Ask (행동/제약)

수행할 명확한 작업 지시, 성능/보안 요구사항

R

Review (평가/개선)

예지 케이스, 테스트 기준 확인 및 스스로 점검 유도

// CHECKPOINTS

- ✓ 모호성 제거 (명확한 제약)
- ✓ 토큰 예산 관리 (간결성)
- ✓ 스타일가이드 준수

prompt_optimization.ts

// ✗ BAD PROMPT

```
const prompt = "이 로그 데이터를 파싱해서 필터링하는 함수 만들어줘.";
```

// ✓ GOOD PROMPT (CLEAR Framework Applied)

```
const prompt = `
[Context] 사용자 로그 데이터를 분석하는 모듈입니다.
[Language] TypeScript, Node.js 환경, date-fns 필수 사용
[Examples]
Input: [{ "date": "2026-05-16", "action": "click" }]
Output: [{ date: Date, isWeekend: true }]
[Ask] 배열에서 주말에 발생한 click 이벤트만 필터링하는
       순수 함수 작성. 성능 최적화(O(n)) 및 타입 정의 포함.
[Review] 작성 완료 후 예지 케이스(빈 배열, 잘못된 날짜
        형식) 처리 방안을 주석으로 요약해줘.
`;
```

반복 개발 & 프롬프트 튜닝

// CORE_ACTIVITIES

핵심 활동

- 작은 배치 크기 (Small Batch Size) 유지
- 회귀 테스트 자동화 및 신속한 검증
- 프롬프트 버저닝 및 실험 결과 로깅
- LLM 평가(Eval) 및 AI와의 페어 프로그래밍

// OUTPUT_DELIVERABLES

산출물

- 체인지로그 (Changelog)
- 프롬프트 라이브러리 (Prompt Library)
- 실험 리포트 및 평가 지표

// RECOMMENDED_TOOLS

추천 도구

- LangSmith / Langfuse
- Weights & Biases (W&B)
- OpenAI Evals
- DVC (Data Version Control)

// VALIDATION_CHECKPOINTS

체크포인트

- 개발 속도와 코드 품질의 균형을 유지하고 있는가?
- 데이터 누락이나 편향 문제가 점검되었는가?
- 정기적인 회고 루프를 통해 개발 과정을 최적화하는가?

개발 검증 (QA)

// CORE_ACTIVITIES

핵심 활동

- 정적분석(SAST) 및 코드 품질 점검
- Unit / Integration / E2E 테스트 자동화 실행
- 계약(Contract), 회귀, 성능 및 보안 테스트 수행
- 테스트 커버리지 측정 및 품질 게이트 통과 검증

// OUTPUT_DELIVERABLES

산출물

- 자동화된 테스트 실행 결과 및 분석 리포트
- 식별된 결함 티켓(Defects) 및 대응 상태
- CI/CD 파이프라인 품질 게이트 통과 증빙 문서

// RECOMMENDED_TOOLS

추천 도구

- SonarQube, Snyk (정적 코드 및 취약점 분석)
- Cypress, Playwright (웹 및 E2E 테스트)
- k6 (부하 및 성능 테스트), Lighthouse
- OWASP ZAP (동적 웹 애플리케이션 보안 진단)

// VALIDATION_CHECKPOINTS

체크포인트

- 릴리스 전 치명적(P0/P1) 결함이 모두 0건인가?
- 정의된 성능 예산(Performance Budget)을 충족하는가?
- 보안 취약점 스캔 결과가 허용 임계치 이하인가?

PHASE 04

배포 & 운영

안정적 출시와 관측 가능성을 확보하고,
신속한 롤백 경로를 보장하여
성공적인 운영 환경을 구축합니다.

```
const phase = {  
  id: "04",  
  status: "DEPLOYING",  
  target: "Production Ops"  
};
```

// OBJECTIVE

목표

- 안정적인 프로덕션 출시
- 시스템 관측 가능성 확보
- 신속한 롤백 경로 보장
- 배포 프로세스 자동화 완성

// REQUIREMENTS

입력 (Input)

- 릴리스 후보(RC) 패키지
- 테스트 및 보안 통과 결과
- 런북(Runbook) 초안
- 품질 게이트 승인 내역

// DELIVERABLES

출력 (Output)

- 구축된 운영환경(Production)
- 모니터링 대시보드 및 알람
- 릴리스 노트(Release Notes)
- 업데이트된 런북/롤백 가이드

// VALIDATION

성공 기준

- 무중단 또는 제한적 중단 배포
- MTTR(평균 복구 시간) 단축
- 사용자 영향 및 오류 최소화
- SLO/SLI 기반 관측 지표 도달

배포 & 운영 환경 구축 (보강)

// CORE_ACTIVITIES

핵심 활동

- IaC(Terraform) 및 환경 구성 (Dev/Stg/Prod)
- 컨테이너화 (Docker/K8s) 적용
- 릴리스 전략 수립 (Blue-Green/Canary/Rolling)
- 관측 가능성 확보 (Logs/Metrics/Traces, SLO/SLI)

// RECOMMENDED_TOOLS

추천 도구

- Terraform, Helm, ArgoCD
- Vercel, AWS, GCP
- Prometheus, Grafana
- Datadog, Sentry

// OUTPUT_DELIVERABLES

산출물

- IaC 리포지토리 및 구성 스크립트
- CD (Continuous Deployment) 파이프라인
- 모니터링 대시보드 및 알람 설정, 롤백 런북

// VALIDATION_CHECKPOINTS

체크포인트

- 시크릿과 구성(Configuration)이 분리되었는가?
- 헬스체크 및 오토스케일링이 적절히 설정되었는가?
- 신속한 롤백 경로가 검증되었는가?

최종 결과 보고서 작성

// CORE_ACTIVITIES

핵심 활동

- 개발 성과 정리 및 KPI/목표지표 측정
- 프로젝트 문서화 (README, API Docs)
- 회고 (Retrospective) 진행
- 이해관계자 최종 보고 및 리뷰

// RECOMMENDED_TOOLS

추천 도구

- Notion / Confluence
- Swagger / OpenAPI
- GitBook
- Mermaid

// OUTPUT_DELIVERABLES

산출물

- 프로젝트 완료 보고서
- 기술 문서 (API 스펙 및 Architecture)
- 릴리스 노트
- KPI 성과 대시보드

// VALIDATION_CHECKPOINTS

체크포인트

- 측정 가능한 구체적인 성과가 명시되었는가?
- 미해결 이슈 및 기술부채가 정확히 기록되었는가?
- 다음 스프린트 또는 운영 백로그와 잘 연결되었는가?

유지보수 & 피드백 사이클 (보강)

// CORE_ACTIVITIES

핵심 활동

- 사용자 피드백 수집 (설문/인터뷰)
- 버그 트래킹 및 이슈 관리
- 지속적 개선(CI) 및 버전 관리(SemVer)
- DevOps Infinity Loop 실천

// OUTPUT_DELIVERABLES

산출물

- 피드백 대시보드
- 백로그 업데이트 리스트
- 핫픽스/패치 릴리스 및 Postmortem 문서

// RECOMMENDED_TOOLS

추천 도구

- Sentry, PagerDuty (에러 및 인시던트 관리)
- GitHub Issues, Linear / Jira (이슈 트래킹)
- Hotjar, Amplitude (사용자 행동 분석)

// VALIDATION_CHECKPOINTS

체크포인트

- SLO/SLA 모니터링이 원활하게 이루어지는가?
- 사용자 NPS 추적 및 회고 주기가 정착되었는가?
- 지속 가능한 기술부채 상환 계획이 수립되었는가?

14단계 통합 프로세스 플로우차트

/src/diagrams/14-steps-integrated-flow.ts



// DELIVERABLE CHAIN

Vision → SRS → Arch/NFR → UI Kit → Test Plan → Dev Env → Increments → Reports → Runbook

핵심 산출물 정리표 (Input → Process → Output)

	STEP	INPUT	PROCESS	OUTPUT
	1. 목적 & 조사	이해관계자 목표, 시장 데이터	문제 정의, 가치 제안 분석	Product Vision, 초기 가설
	2. 1차 개발 설계	Product Vision, 도메인 지식	아키텍처 스케치, 기술 옵션 비교	1차 아키텍처, ADR, 인터페이스
	3. SRS 명세 작성	1차 설계안, 비즈니스 룰	기능/비기능 요구사항, 리스크 분석	SRS v1, MoSCoW 백로그
	4. 보안 & NFR	SRS v1, 아키텍처 후보	위협모델링, 성능예산 및 확장성 설계	보안 설계 문서, NFR 스펙
	5. UI/UX 화면	SRS v1, 페르소나	IA 설계, 와이어프레임, 컴포넌트	화면흐름도, 디자인 시스템
	6. AI 자료 정리	NFR 스펙, UI/UX 디자인	AI 컨텍스트 패키징, API 스펙 정리	AI 브리프, 데이터 계약서
	7. TDD 계획	AI 브리프, 기능 요구사항	테스트 전략 수립, Gherkin AC 작성	테스트 계획, 스펙/피쳐 파일
	8. 환경 세팅	아키텍처, 툴 스택 옵션	레포 초기화, CI/CD 스캐폴딩	부트스트랩 레포, CI 인프라
	9. 바이브 코딩	테스트 케이스, AI 브리프	프롬프트 루프, AI 페어 프로그래밍	기능 증분, 스니펫, PR 리뷰
	10. 반복 & 튜닝	기능 증분, 실행 결과	프롬프트 최적화, 배치 단위 개발	Prompt 라이브러리, 체인지로그
	11. 개발 검증	통합 코드베이스, 테스트 스펙	정적분석(SAST), E2E/보안 테스트	테스트 리포트, 게이트 패스 증빙
	12. 환경 구축	릴리스 후보(RC) 패키지	IaC 구성, 릴리스/관측 전략 셋업	운영환경(Prod), 런북, 대시보드
	13. 결과 보고서	배포 결과, KPI 모니터링 수치	성과 정리, API/아키텍처 문서화	완료 보고서, 릴리스 노트
	14. 유지보수	운영 서비스, 사용자 로그	사용자 피드백 수집, 버그 핫픽스	피드백 대시보드, 백로그 갱신

추천 도구 스택 (검수 보강판)

/src/tools/recommended-stack.ts | v0는 UI 프로토타입으로 이동, Codex/Gemini 계열 추가

기획/리서치

Lean Canvas Miro/FigJam
Google Trends GA4/Mixpanel
Typeform User Interviews

설계/문서/UI

Figma Storybook diagrams.net
Notion/Confluence OpenAPI/Swagger
v0 (UI prototype)

AI 코딩/에이전트

Cursor GitHub Copilot Claude Code
OpenAI Codex Gemini Code Assist
Gemini CLI

테스트/QA 보안

Jest/Vitest pytest Playwright
Cypress Pact k6
SonarQube OWASP ZAP Snyk/Trivy

배포/운영

Docker Kubernetes Vercel
AWS/GCP/Azure Terraform/OpenTofu
GitHub Actions Argo CD

모니터링/인시던트

OpenTelemetry Datadog Sentry
Prometheus Grafana PagerDuty
Jira Service Management

검수 메모

v0는 범용 코딩 에이전트가 아니라 UI/프론트엔드 프로토타이핑 성격이 강하므로 AI 코딩 칸에서 분리했습니다. Codex와 Gemini 계열은 범용 AI 코딩/에이전트 도구로 보강했습니다.

AI 바이트 코딩 장점 & 주의사항

장점 (PROS)

// ADVANTAGES_AND_OPPORTUNITIES

- 1 개발 속도 10배 향상
- 2 진입장벽 대폭 낮춤
비개발자도 로직 구현 및 수정 가능
- 3 빠른 프로토타이핑 / MVP 검증
- 4 반복적이고 단순한 코드 자동화
- 5 최신 라이브러리 및 디자인 패턴 활용
- 6 문서화(주석, README 등) 자동 생성

주의사항 (CONS)

// RISKS_AND_VULNERABILITIES

- 1 코드 품질 검증 필수 (QA/테스트 누락 시 위험)
- 2 보안 취약점 발생 위험
Hallucination(환각)으로 인한 잘못된 코드 생성
- 3 AI 의존성 증가 및 개발자 디버깅 역량 저하
- 4 AI 생성 코드의 라이선스 및 저작권 이슈
- 5 비즈니스 개인정보 및 기밀 유출 위험
- 6 전체 시스템 아키텍처의 일관성 부족

전통적 개발 vs AI 바이브 코딩 비교

	전통적 개발 (Traditional)	AI 바이브 코딩 (Vibe Coding)	우위 (Winner)
개발 속도	보통 (수동 타이핑 및 구조 설계)	매우 빠름 (프롬프트 기반 코드 생성)	AI
초기 학습 곡선	가파름 (언어/프레임워크 숙달 필요)	완만함 (자연어 프롬프트 활용)	AI
코드 품질 일관성	높음 (팀 컨벤션 및 룰 준수)	검증 필요 (Hallucination 가능성)	전통
프로토타이핑	느림 (기본 보일러플레이트 작성)	매우 빠름 (즉각적인 MVP 시각화)	AI
디버깅 난이도	보통 (로직 흐름 파악 용이)	어려움 (블랙박스형 생성 코드)	전통
보안/검증 부담	낮음 (사전 보안 설계 반영)	높음 (AI 생성 코드 취약점 검토 필수)	전통
비용 효율성	높은 인건비 및 시간 소요	저비용 빠른 산출 (도구 구독료 위주)	AI
팀 구성	다수의 시니어/주니어 분업 필요	소수 정예 + AI (1인 다역 가능)	AI

💡 결론: 속도와 진입장벽은 AI 우세, 안정성과 일관성은 전통 우세 → 하이브리드 (People + AI + Process) 접근이 최적

성공을 위한 Best Practices



1

Plan First, Code Later (계획 우선)

바이브 코딩 전 명확한 아키텍처와 요구사항 설계가 필수적입니다.



2

AI는 도구, 판단은 사람 (Human-in-the-loop)

최종 검토와 기술적 의사결정은 항상 개발자가 주도해야 합니다.



3

작은 단위로 반복 (Small Iterations)

한 번에 큰 기능을 요청하지 말고, 작은 단위로 쪼개어 검증합니다.



4

테스트가 곧 명세 (TDD First)

테스트 코드를 먼저 작성하여 AI가 목표 삼을 명확한 기준을 제시합니다.



5

ADR로 결정 기록 (Document Decisions)

AI와의 문맥 공유를 위해 주요 아키텍처 결정을 문서로 남깁니다.



6

보안은 시작부터 (Shift-Left Security)

설계 단계부터 취약점을 점검하고, 프롬프트에 제약사항을 포함합니다.



7

회고와 측정 (Retrospective + Metrics)

품질 지표와 리드타임을 측정하여 AI 프롬프트와 프로세스를 개선합니다.

Common Pitfalls Top 5



1

⚠ 실수

AI에게 통째로 맡기기

✓ 해결

작업을 작게 쪼개고 단계별 검증



2

⚠ 실수

프롬프트에 컨텍스트 부족

✓ 해결

CLEAR 프레임워크 + 기존 코드 컨벤션 명시



3

⚠ 실수

테스트 없이 다음 기능으로

✓ 해결

TDD + CI 자동화 게이트 도입



4

⚠ 실수

보안/성능 검증 생략

✓ 해결

정적분석+SAST를 PR 필수 단계로 추가



5

⚠ 실수

코드 리뷰 패스

✓ 해결

AI 생성 코드도 휴먼 리뷰 의무화

케이스 스터디 — To-Do SaaS MVP

☰ 14단계 적용 타임라인 요약

- [P1] 목적/1차설계: 개인용 할일 관리 MVP 정의
- SRS 작성: 기본 CRUD 및 인증 요건 문서화
- [P2] 보안/UI 설계: Supabase RLS 규칙, Figma 화면
- AI/TDD 준비: API 스펙 브리프, Jest 케이스
- 환경 세팅: Next.js + Cursor IDE 스캐폴딩
- [P3] 바이트 코딩: 컴포넌트 15개 2일만에 생성
- 반복/튜닝: 상태관리 버그 프롬프트로 해결
- 검증(QA): Cypress E2E 테스트 100% 통과
- [P4] 배포/운영: Vercel 원클릭 배포 완료
- 보고/유지보수: 사용자 피드백 백로그 등록 (v1.1)

stack.json prompt.md result.log

```
// 1. Tech Stack Definition
const techStack = [ "Next.js", "TypeScript", "Supabase", "Vercel" ];

// 2. Vibe Coding AI Prompt (CLEAR Framework)
> YOU:
"Context: 할일 관리 SaaS MVP 개발
Language: Next.js(App Router), TypeScript, 인라인 스타일
Ask: Supabase를 연동하여 JWT 기반 로그인과 Todo CRUD 컴포넌트를 구현해줘.
Review: 작성 전 Jest 테스트 코드를 먼저 제시하고 통과 가능하도록 작성해."
```

```
// 3. Execution Result
✓ 12 Components generated & reviewed
✓ 45 Test cases passed (85% coverage)
➔ MVP Shipped in 2 weeks (Traditional est: 6 weeks)
```

📊 핵심 결과: 개발 기간 70% 단축 | 테스트 커버리지 85% | PO 버그 0건

체크리스트 요약 (Quick Reference)

// PHASE 01

기획 & 조사

- ☐ 1. 목적/시장 조사
- ☐ 2. 1차 설계
- ☐ 3. SRS 작성

// PHASE 02

설계 & 준비

- ☐ 4. 보안/NFR
- ☐ 5. UI/UX
- ☐ 6. AI 브리프
- ☐ 7. TDD
- ☐ 8. 환경 세팅

// PHASE 03

개발 & 검증

- ☐ 9. 바이브 코딩
- ☐ 10. 반복/튜닝
- ☐ 11. QA

// PHASE 04

배포 & 운영

- ☐ 12. 배포
- ☐ 13. 보고서
- ☐ 14. 유지보수

💡 Tip: 각 단계 완료 시 산출물 확인 → Gate 통과 → 다음 단계로

결론 & 핵심 메시지

Plan First, Code with AI, Verify Always



Plan First

명확한 계획과 SRS(요구사항 명세서) 없이 무작정 코드 작성을 시작하지 않습니다.



Code with AI

AI는 강력한 페어 프로그래머일 뿐, 시스템의 최종 자율 의사결정자가 아닙니다.



Verify Always

모든 AI 산출물은 철저한 테스트, 휴먼 리뷰, 보안 검증을 반드시 거쳐야 합니다.

“ AI 바이브 코딩은 도구일 뿐, 소프트웨어 공학의 원칙은 변하지 않습니다. ”

개발 속도

10x ↑

진입장벽

70% ↓

품질 검증 필수도

100%

v2 보강 부록

원본 14단계 프로세스를 실행 가능한 개발 운영 플레이북으로 보강

Gate, Security, AI Governance, Ops 보강

- 원본 33페이지는 그대로 유지하고, 실제 팀 적용에 필요한 판정 기준과 증빙 체계를 추가했습니다.
- 핵심 보강 방향은 각 단계의 산출물을 Owner / Entry Criteria / Exit Criteria / Evidence / Approver / Metric으로 달는 것입니다.
- AI 기반 개발의 속도 장점은 유지하되, 보안 □ 라이선스 □ 개인정보 □ 운영 리스크를 릴리스 게이트에서 검증하도록 구성했습니다.

적용 원칙

Plan First, Code with AI, Verify Always + Evidence Before Approval

모든 AI 산출물은 테스트, 보안 검증, 휴먼 리뷰, 릴리스 승인 증빙 없이는 운영 배포하지 않습니다.

Gate 강화 기준

G1/G2/G3를 “ 이름 있는 마일스톤” 에서 “ 증빙 기반 승인 절차” 로 전환

Gate	위치	필수 증빙	승인 기준
G1 Requirements Freeze	Step 03 이후	PRD/SRS, AC, RTM, 리스크 레지스터, 개인정보/규제 체크, 변경관리 규칙	핵심 요구사항 모호성 제거, 테스트 가능성 확보, 이해관계자 승인
G2 Design Readiness	Step 08 이후	C4, ADR, 데이터 모델, API 계약, 위협모델, NFR 시나리오, CI Green	개발 착수 전 구조/보안/테스트/환경 준비 완료
AI Governance Gate	Step 09 전	승인 모델/도구 목록, 입력 금지정보, 프롬프트 로그 정책, AI 리뷰 체크리스트	민감정보 유출 및 무검증 AI 코드 반입 방지
G3 Release Approval	Step 11 이후	P0/P1 0건, 테스트 리포트, SAST/DAST/SCA, SBOM, 성능/접근성, UAT	운영 배포 가능한 품질 ▯ 보안 ▯ 성능 기준 충족
Operational Readiness	Step 12 전후	런북, 알람, 온콜, 백업/복구 테스트, 마이그레이션/롤백, Feature Flag	장애 대응과 되돌리기 경로가 실제로 검증됨

각 Step 실행 템플릿

모든 단계에 동일한 운영 필드를 추가해 누락과 해석 차이를 줄임

Step Template

- Owner: 산출물 작성 책임자
- Reviewer: 기술/보안/제품 검토자
- Approver: 다음 단계 진입 승인자
- Entry Criteria: 시작 전 필요한 입력물
- Exit Criteria: 단계 완료 조건
- Required Evidence: 저장해야 할 증빙 링크/파일
- Metric: 단계 성공을 측정할 수치
- Rollback/Change Rule: 변경 발생 시 되돌림/재승인 규칙

예시: Step 11 개발 검증

- Owner: Tech Lead / QA Lead
- Entry: RC 빌드, 테스트 스펙, 위협모델, 릴리스 노트 초안
- Exit: P0/P1 0건, 필수 테스트 통과, 보안 임계치 통과
- Evidence: CI URL, 커버리지, SAST/DAST/SCA 결과, ZAP 리포트
- Approver: Product Owner + Security Reviewer + Tech Lead
- Metric: defect density, coverage, p95 latency, vulnerability threshold

보안 보강 체크리스트

Shift-left Security를 PR/릴리스 게이트에서 닫히는 검증 항목으로 구체화

영역	필수 점검 항목	권장 증빙
AI 특화 보안	prompt injection, tool misuse, 코드/시크릿 유출 abuse case 테스트	AI threat model, red-team prompt 결과
공급망 보안	SCA, SBOM, lockfile 고정, typosquatting, 라이선스, 컨테이너 이미지 스캔	Snyk/Dependabot/Tri vy/Cycl oneDX 리포트
인증/인가	역할/권한 매트릭스, 테넌트 격리, RLS 테스트, JWT/session 만료 회전, MFA 기준	AuthZ 테스트 결과, RLS 정책 테스트
API/Input	schem validation, XSS, SQLi/NoSQLi, SSRF, CSRF, CORS, file upload, rate limit	E2E/DAST/API 보안 테스트 결과
Secrets/Data	secrets scan, KMS/키 회전, 암호화, 개인정보 마스킹, 보존/삭제 정책	secret scan 결과, 데이터 분류표
운영 보안	감사 로그, 보안 이벤트 알림, 침해 대응 런북, 패치 SLA, 백업 복구 훈련	IR runbook, 복구 리허설 기록

AI 사용 정책

AI Vibe Coding의 속도는 유지하되 입력 □ 도구 □ 산출물 경계를 명확히 함

입력 통제

- 개인정보, 고객 데이터, 비밀키, 계약서, 미공개 재무정보 입력 금지
- 필요 시 샘플/마스킹 데이터만 사용
- 프롬프트 로그 보관 범위와 삭제 주기 명시
- 외부 모델 전송 가능 데이터 등급 정의

도구 통제

- 승인된 모델, IDE, MCP/플러그인, CLI 목록 관리
- 파일 쓰기, 네트워크, 배포, 결제 등 외부 영향 작업은 승인 필요
- 에이전트 작업 로그와 변경 파일 추적
- AI 도구별 데이터 보존/학습 사용 정책 확인

산출물 통제

- AI 생성 코드는 휴먼 리뷰 필수
- 라이선스/SBOM/SCA 통과 전 병합 금지
- 중요 로직은 테스트와 ADR로 근거 기록
- 보안 □ 성능 제약은 프롬프트와 PR 템플릿 모두에 반영

요구사항 추적성 매트릭스

요구사항이 테스트와 릴리스 증빙까지 끊기지 않도록 연결

Req ID	요구사항/AC	테스트	PR/Commit	릴리스 증빙
REQ-AUTH-001	사용자는 이메일/Password로 로그인할 수 있다. 실패/잠금/만료 케이스 포함	unit: auth service, e2e: login flow security: brute force	PR-123, commit abc123	CI green, UAT sign-off, release note
REQ-AUTHZ-002	사용자는 본인 리소스만 조회/수정할 수 있다	RLS/AuthZ matrix, tenant isolation test	PR-127	권한 테스트 리포트, DAST 결과
NFR-PERF-001	주요 API p95 latency 300ms 이하	k6 load test, APM baseline	PR-140	성능 리포트, SLO 대시보드
SEC-DATA-001	개인정보는 저장/전송 시 암호화하고 로그에 남기지 않는다	secret scan, log inspection, data masking test	PR-145	보안 리뷰 승인, 데이터 분류표

운영 규칙

새 요구사항이나 AC가 추가되면 테스트 ID와 릴리스 증빙이 함께 추가되어야 합니다. 추적 링크가 없는 요구사항은 [G3 Release Approval](#) 을 통과할 수 없습니다.

운영 준비도 보강

배포 자동화뿐 아니라 장애 대응과 복구 가능성을 검증

영역	Ready 기준	증빙
배포/롤백	Blue-Green/Canary/Rolling 전략, 즉시 롤백 절차, Feature Flag 준비	배포 리허설, 롤백 리허설 기록
데이터 변경	DB migration/rollback, 백필 전략, 데이터 정합성 검증	마이그레이션 계획서, dry-run 결과
관측성	Logs/Metrics/Traces, SLO/SLI, 알람 임계치, 대시보드	Grafana/Datadog/Sentry 링크
인시던트 대응	등급 체계, 온콜, 에스컬레이션, 커뮤니케이션 템플릿	IR runbook, 모의훈련 기록
복구/연속성	backup/restore, DR, RPO/RTQ 키 회전, 비상 권한 회수	복구 테스트 결과, 접근권한 점검표

적용 로드맵

문서 개정 후 실제 팀 프로세스에 반영하는 순서

1주차: 기준 확정

- G1/G2/G3 체크리스트 확정
- Step 템플릿에 Owner/Approver 추가
- AI 입력 금지정보와 승인 도구 목록 공지
- PR 템플릿에 테스트/보안 증빙 필드 추가

2-3주차: 자동화

- SAST/SCA/secrets scan CI 필수화
- SBOM생성과 라이선스 검증 추가
- RTM 표준 양식 도입
- 릴리스 체크리스트와 런북 템플릿 적용

4주차 이후: 운영화

- Operational Readiness Review 정례화
- 백업/복구 및 롤백 리허설 수행
- 보안 이벤트 알림과 패치 SLA 운영
- 회고에서 defect density, lead time, SLO를 개선 지표로 사용

v2 결론: 빠른 AI 개발 흐름은 유지하되, 승인 게이트는 반드시 증빙 기반으로 닫습니다.

Thank You

질문이 있으신가요? ■

```
// Q&A SESSION OPEN  
  
> await user.askQuestion();  
> Processing feedback...  
> -
```

```
const resources = {  
  docs: 'github.com/vibe-coding',  
  cursor: 'cursor.com',  
  copilot: 'github.com/features/copilot',  
  claude: 'claude.ai/code'  
};
```